

GPU COMPUTING

- The trend is towards heterogeneous computing in which CPUs and GPUs work together for different parts of a computation.
- The use of GPU as a general purpose unit for solving a given problem, unrestricted to the graphical context.

112

HIGH-PERFORMANCE COMPUTING

- **High-Performance Computing (HPC)** is concerned with developing tools, algorithms, and applications that can make optimal use of a given hardware environment.
- the most important measure for the performance of a computing device in HPC is the number of floating point operations per second possible-FLOPS/s
- HPC rely on floating point operations
 - defined in the IEEE 754 standard - most known standards in numerical computing.
 - single precision and double precision floating point numbers

Name	Peak Performance (GFlop/s)	Notes
Intel Xeon Platinum 8280M (28 Cores)	1,612.8	A fast workstation CPU
Raspberry Pi 4 Model B	24	A very cheap and fun to code for ARM CPU
Nvidia RTX 4090	73,000	Single precision peak for Nvidia's new GPU generation
PS5 GPU	10,280	Single precision peak of the new GPU for the PS5
XBOX Series X GPU	12,500	Single precision peak of the new GPU for the XBOX Series X

113

PARALLEL COMPUTING

- **Parallel processing**, or **parallel computing**, is a type of computing that relies on two or more processors to accomplish different subsets of an overall computing task.
 - Older generation computers could only run one program at a time, often taking hours to complete a task.
 - GPUs' parallel processing function performs many calculations or tasks simultaneously, making them faster and more efficient than CPUs in older computers.

114

BASIC CONCEPTS

- **Concurrency** - is a property of a program (at design level) where two or more tasks can be in progress simultaneously
- **Parallelism** - is a run-time property where two or more tasks are being executed simultaneously

“being in progress” is different from “being executed”

Parallelism requires concurrency, but concurrency does not require parallelism.

- **Parallel computing** - is the act of solving a problem of size n by dividing its domain into $k \geq 2$ (with $k \in \mathbb{N}$) parts and solving them with p physical processors, simultaneously.
 - **Data-parallel problem** - PD is a data-parallel problem if D is composed of data elements and solving the problem requires applying a kernel function $f(\dots)$ to the whole domain – GPU
 - **Task-parallel problem** - PD is a task-parallel problem if D is composed of functions and solving the problem requires applying each function to a common stream of data S - CPU

<https://www.researchgate.net/publication/256495766>

115

ACTIVITY

- Relation between Deep Learning and GPU
- What is a GPU?
- Why do we need a GPU?
- When to use GPU/CPU?
- Differences between GPU and CPU

116

WHAT DOES GPU STAND FOR?

- Graphics processing unit, a specialized processor originally designed to accelerate graphics rendering.
- Can process many pieces of data simultaneously, making them useful for machine learning, video editing, and gaming applications.
- May be integrated into the computer's CPU or offered as a discrete hardware unit.

117

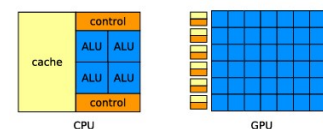
WHAT'S THE DIFFERENCE BETWEEN A GPU AND A CPU?

- GPUs work in conjunction with a server's CPU to accelerate application and processing performance.
- The CPU offloads compute intensive portions of the application to the GPU, which processes large blocks of data at one time rather than sequentially to boost overall performance in a server environment. By running smaller tasks concurrently, GPUs harness the processing power needed for performance-hungry workloads.

118

WHAT'S THE DIFFERENCE BETWEEN A GPU AND A CPU?

- CPUs and GPUs share a similar design, including a similar number of cores and transistors for processing tasks:
 - CPUs are more general-purpose in their functions than GPUs.
 - CPUs tend to use fewer cores and perform their tasks in a linear order,
 - GPUs tend to be focused on a singular, specific computing task (graphics processing or machine learning).
 - GPUs have hundreds—even thousands—of cores, enabling the parallel processing that drives their lightning-fast processing capabilities.
- CPUs are the heart and brain of a computer system or device. They receive general instructions or requests regarding a task from a program or software application.
- A GPU, on the other hand, has a more specific task:
 - typically involving the processing of high-resolution images and videos quickly.
 - To accomplish their task, perform complex mathematical calculations required for rendering graphics or other compute intensive functions.



119

CPU versus GPU



120

GPU COMPUTING

CPU	GPU
A smaller number of larger cores (up to 24)	A larger number (thousands) of smaller cores
Low latency	High throughput
Optimized for serial processing	Optimized for parallel processing
Designed for running complex programs	Designed for simple and repetitive calculations
Performs fewer instructions per clock	Performs more instructions per clock
Automatic cache management	Allows for manual memory management
Cost-efficient for smaller workloads	Cost-efficient for bigger workloads

121

GPU COMPUTING

GPUs are becoming the modern general processor

- Graphics processing unit (GPU) has become one of the most important types of computing technology for personal and business computing.
- Designed for parallel processing, is used in a wide range of applications, including graphics and video rendering. Although they're best known for their capabilities in gaming and are becoming more popular for use in creative production and artificial intelligence (AI).
- Originally designed to accelerate the rendering of 3D graphics.
- Then, became more flexible and programmable, enhancing their capabilities. This allowed graphics programmers to create more interesting visual effects and realistic scenes with advanced lighting and shadowing techniques. Other developers also began to tap the power of GPUs to dramatically accelerate additional workloads in high performance computing (HPC), deep learning, and more.

122

DIFFERENT TYPES OF GPUS

- **Discrete GPUs**, or dGPUs, are graphics processors that are separate from a device's CPU, where information is taken in and processed, allowing a computer to function. Discrete GPUs are typically used in advanced applications with special requirements, such as video editing, content creation or high-end gaming. They are distinct chips with connectors to separate circuit boards that are usually attached to the CPU using an express slot. One of the most widely used discrete GPUs is the Intel Arc brand, which was built for the PC gaming industry.
- **Integrated GPU**, or iGPU, is built-in to a computer or device's infrastructure and typically slotted in next to the CPU. Designed in the 2010s by Intel, integrated GPUs became more popular as manufacturers such as MSI, ASUS and Nvidia noticed the power of combining GPUs with CPUs rather than requiring users to add GPUs via a PCI express slot themselves. Today, they remain a popular choice for laptop users, gamers and others running compute intensive programs on their PCs.
- **Virtual GPUs**: have the same capabilities as discrete or integrated GPUs, but without the hardware. They are simply a software-based version of a GPU built for a cloud instance and can be used to run the same workloads. Also, since they have no hardware, they are simpler and cheaper to maintain than their physical counterparts.

123

GPU COMPUTING STANDARDS AND LANGUAGES

Different ways to develop software for GPU accelerators., like:

- **Nvidia Cuda** is arguably the most advanced development platform for GPU computing. Cuda can be used from C/C++, Fortran, Python, Matlab, Julia, and others.
- **OpenCL** is the open heterogeneous computing standard from the Khronos Group. It does not only support GPU accelerators but also CPUs, FPGA's and essentially any type of device that provides an OpenCL interface.
- **Sycl** is a modern heterogeneous compute standard from the Khronos Group. It is built on standard C++ and quickly gaining usage share in the HPC world. **OneAPI** is Intel's contribution to the development of an Industry wide standard for heterogeneous computing. It is essentially an extension of Sycl and mostly compatible to Sycl. Intel is in the process of building up a complete software ecosystem around OneAPI and supports open-source OneAPI compiler development based on the LLVM platform.
- **OpenACC** is a standard for compiler pragma's that support offloading of computations to accelerator devices. It supports C, C++ and Fortran. It is mainly pushed by Cray in partnership with Nvidia.
- **OpenMP** was originally a standard for easily developing parallel multithreaded applications on CPU platforms. It provides simple compiler pragmas to enable parallel execution of source code segments in C, C++ and Fortran.

124

HARDWARE FOR GPU COMPUTING

- **Nvidia** - All modern Nvidia GPUs are well suited for GPU computing. Nvidia cards support all major heterogeneous compute standards. The top of the line GPU accelerator from Nvidia is the A100, which has a double precision peak performance of 10 TFlops and a single precision performance of 20 TFlops.
- **AMD** - support comes through the Rocm platform (<https://rocm.docs.amd.com/en/latest/>). This is supported for cards up to the Vega architecture, but not officially for RDNA, the current generation of accelerators. AMD's data center GPU compute solutions are still based on Vega. AMD announced CDNA, a new architecture for data centers for compute purposes. While AMD is closing up to Nvidia in terms of performance it's compute platform support is much less straight forward than Nvidia.
- **Intel** - Intel traditionally has very good support for OpenCL on its CPU and GPU devices. However, Intel's current GPUs are not competitive for compute applications (though their performance is not bad for certain applications). This is changing with Intel's Xe generation of GPU acceleration that starts coming out in 2020. Intel Xe will power several planned Exascale architectures and most likely well support OpenCL, OpenMP and OneAPI.

125

GPU COMPUTING IN PYTHON

- Python has strong support for OpenCL and Cuda.:
 - The **PyCUDA** and **PyOpenCL** libraries are mature packages that allow to write GPU kernels in C and to launch them from Python.
 - With **PyOpenCL** and a CPU OpenCL drivers we can also launch compute kernels for CPUs. Since PyOpenCL knows native SIMD types this is a great way to write low-level SIMD accelerated Code for CPUs.

```
import pycuda.autoint
import pycuda.driver as drv
import numpy

from pycuda.compiler import SourceModule
mod = SourceModule("""
__global__ void multiply_them(float *dest, float *a, float *b)
{
    const int i = threadIdx.x;
    dest[i] = a[i] * b[i];
}
""")

multiply_them = mod.get_function("multiply_them")

a = numpy.random.randn(400).astype(numpy.float32)
b = numpy.random.randn(400).astype(numpy.float32)

dest = numpy.zeros_like(a)
multiply_them(
    drv.Out(dest), drv.In(a), drv.In(b),
    block=(400,1,1), grid=(1,1))

print(dest-a*b)
```

[Back to top](#)

```
#!/usr/bin/env python
import numpy as np
import pyopencl as cl

a_np = np.random.randn(50000).astype(np.float32)
b_np = np.random.randn(50000).astype(np.float32)

ctx = cl.create_some_context()
queue = cl.CommandQueue(ctx)

mf = cl.mem_flags
a_g = cl.Buffer(ctx, mf.READ_ONLY | mf.COPY_HOST_PTR, hostbuf=a_np)
b_g = cl.Buffer(ctx, mf.READ_ONLY | mf.COPY_HOST_PTR, hostbuf=b_np)

prg = cl.Program(ctx, """
__kernel void sum(
    __global const float *a_g, __global const float *b_g, __global float *res_g)
{
    int gid = get_global_id(0);
    res_g[gid] = a_g[gid] + b_g[gid];
}
""").build()

res_g = cl.Buffer(ctx, mf.READ_ONLY, a_np.dtype)
prg.run(queue, a_np.shape, None, a_g, b_g, res_g)

res_np = np.empty_like(a_np)
cl.enqueue_copy(queue, res_np, res_g)

# Check on CPU with Numpy:
print(res_np - (a_np + b_np))
print(np.linalg.norm(res_np - (a_np + b_np)))
assert np.allclose(res_np, a_np + b_np)
```

[Back to top](#)
https://tбетcke.github.io/hpc_lecture_notes/gpu_introduction.html

126

PARALLEL COMPUTING IN PYTHON

- Current computers are highly parallel systems:
 - Each CPU consists of multiple CPU cores, and within each CPU cores there are vector units that allow the parallel execution of certain operations.
 - GPU accelerators are highly parallel devices themselves. If we move to larger compute clusters then there is also a level of parallelism between the individual hardware nodes.
- Computer architectures can be classified by using Flynn's taxonomy:
 - SISD** (single instruction single data stream) can only perform one instruction to one data stream at a time.
 - SIMD** (single instruction multiple data streams) can handle only one instruction
 - MISD** (multiple instruction single data stream) can handle different tasks over the same stream of data.
 - MIMD** (multiple instruction multiple data streams) is the most flexible architecture. It can handle one different instruction for each data stream and can achieve any type of parallelism.
- Memory architectures and organizations
 - Distributed memory**, each node has its own memory architecture and it is completely independent from other nodes. Communication is based on messages between nodes through a network. The network plays a important role and its topology is different depending on the context.
 - Shared memory organization**, processors communicate through a common bank of global memory, not needing explicit messages as in a distributed memory scheme. Two architectures are mostly used: Uniform Memory Access or UMA consists of a shared memory in which the access time for any processor takes the same amount of time no matter the data location; also known as Symmetric Multi-Processors or SMP. The main disadvantage of UMA is the low scalability when increasing the number of processors. This occurs because of the single memory controller shared for all processors

127

PYTHON HPC TOOLS

Python has an incredible ecosystem of tools for scientific computing:

- **Jupyter Notebook** - Jupyter is a key part of the Python ecosystem. It allows the creation of Jupyter Notebook documents that mix executable code, descriptions, figures and formulas in a single file that can be viewed and edited inside a web browser.
- **Numpy and Scipy** - Numpy and Scipy are key tools for any scientific Python installation. Numpy defines a fast array data type and provides a huge amount of operations on this type including all common linear algebra operations.
- **Numba** - Numba is a tool for the just-in-time compilation of Python functions. Python itself is a slow language. Each operation has considerable overhead from the Python interpreter, making especially time-critical for-loops inefficient in Python. Numba allows to just-in-time compile Python functions into direct machine code that needs not access the Python interpreter.
- **Matplotlib** - Matplotlib is the standard on Python for data visualization in two dimensions. It is incredibly feature rich.
- **Dask** - Dask provides a powerful environment to specify complex calculations as graph that is then optimally executed on a given parallel environment. It allows scalability for algorithms from a single desktop up to a HPC systems with thousands of nodes.
- **Pandas** - Pandas is the standard data-analysis library for Python. It has efficient data structures and tools to handle and process large-scale data sets in Python.
- **Tensorflow, PyTorch, and scikit-learn** – Python is the preferred environment for machine learning. We will not go into the various tools as part of this module, but just mention some of them for completeness, in particular Tensorflow, PyTorch and scikit-learn.
- Other tools: **mpi4py** for data communication on clusters using the MPI standard, **petsc4py**, an interface to the widely used Petsc library for parallel sparse matrix operations, or **FEniCS**, a powerful PDE solution package using the finite element method.

128

SIMD (SINGLE-INSTRUCTION-MULTIPLE-DATA)

- SIMD (Single-Instruction-Multiple-Data) operations using vector registers. CPU core has internal registers that allow the execution of a command on several arguments within a single CPU cycle.
- Assuming a SIMD register and a SIMD register in the CPU, each of them holding four pieces of data. Then the addition of the four pieces of data can be done all at once within a single CPU cycle.
- SIMD instructions are a very low-level tool, which we cannot use directly in Python. However, several libraries provide functionality that can take advantage of SIMD instructions, in particular:
- Numpy benefits from SIMD if the underlying BLAS library uses SIMD instructions.
- The Numexpr library allows the generation of complex transformation over arrays that will be translated to fast SIMD instructions under certain conditions.
- The Numba Just-In-Time compiler for Python code can auto-compile certain for-loops into accelerated SIMD code.

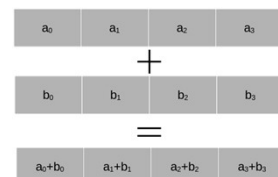
```
import numpy as np

n = 1000000
a = np.random.randn(n)
b = np.random.randn(n)

c = np.empty(n, dtype='float64')

for index in range(n):
    c[index] = a[index] + b[index]
```

$c = a + b$



129

CPU versus GPU

Aspecto	CPU (Unidade Central de Processamento)	GPU (Unidade de Processamento Gráfico)
Função Principal	Executa tarefas gerais e instruções de programas.	Processa gráficos e realiza cálculos paralelos.
Arquitetura	Poucos núcleos poderosos (2 a 64 núcleos).	Muitos núcleos menores e mais simples (centenas a milhares).
Desempenho em Tarefas	Ideal para tarefas complexas e sequenciais.	Ideal para tarefas paralelas e processamento em larga escala.
Tipo de Processamento	Processamento sequencial e de instruções complexas.	Processamento paralelo e operações simultâneas em muitos dados.
Uso Comum	Execução de sistemas operacionais, aplicativos e tarefas variadas.	Renderização de gráficos, processamento de vídeo, e cálculos científicos.